

Mục lục

Lời nói đầu	4
1 Lớp 0 — Phần cứng và Cảm biến	5
1.1 Tổng quan vai trò của lớp cảm biến trong hệ thống thị giác tốc độ cao . .	5
1.2 Global Shutter và Rolling Shutter	5
1.2.1 Nguyên lý Rolling Shutter	5
1.2.2 Nguyên lý Global Shutter	5
1.2.3 So sánh định lượng	6
1.3 Motion Blur và yêu cầu thời gian phơi sáng	6
1.3.1 Ràng buộc thiết kế	6
1.4 Giới hạn photon và bài toán ánh sáng	7
1.5 Hệ thống chiếu sáng hồng ngoại (IR Illuminator)	7
1.5.1 Nguyên lý hoạt động	7
1.5.2 So sánh 850nm và 940nm	7
1.5.3 Tương tác với vật liệu phản quang	7
1.6 Tổng hợp các ràng buộc thiết kế	8
1.7 Kết luận chương	8
2 Lớp 2 — Video Codec và Cơ chế Trigger	9
2.1 Giới thiệu	9
2.2 Tách biệt xử lý CPU và AI Inference	9
2.2.1 Mô hình pipeline truyền thống	9
2.2.2 Mô hình pipeline tối ưu	9
2.2.3 Mô hình hàng đợi (Queueing Model)	10
2.3 Video Codec và cấu trúc GOP	10
2.3.1 Group of Pictures (GOP)	10
2.3.2 Tối ưu hóa decode	10
2.4 Cơ chế ROI Trigger	10
2.4.1 Định nghĩa ROI	10
2.4.2 Trigger dựa trên biến đổi cường độ	11
2.4.3 Trigger dựa trên optical flow	11
2.5 Frame Selector và bài toán tối ưu	11
2.5.1 Định nghĩa hàm chất lượng frame	11
2.5.2 Đo độ sắc nét	11

2.5.3	Do motion blur	11
2.5.4	Bài toán chọn frame	12
2.6	Giảm tải NPU và tối ưu hệ thống	12
2.6.1	Hiệu suất tính toán	12
2.6.2	Tối ưu hóa xác suất trigger	12
2.7	Song song hóa đa luồng	12
2.8	Kết luận chương	13
3	Lớp 3 — Core AI Inference	14
3.1	Tổng quan	14
3.2	Bài toán phát hiện đối tượng	14
3.2.1	Mô hình hóa	14
3.2.2	Anchor-free detection	14
3.2.3	IoU Loss	14
3.3	Pipeline nhận dạng chuỗi ký tự	15
3.3.1	Bài toán	15
3.3.2	CTC (Connectionist Temporal Classification)	15
3.3.3	Ưu điểm	15
3.4	Ước lượng tốc độ	15
3.4.1	Chuyển đổi tọa độ	15
3.4.2	Tốc độ	16
3.5	Tối ưu hóa trên NPU	16
3.5.1	Quantization	16
3.5.2	Throughput	16
3.6	Kết luận	16
4	Lớp 5 — Violation Logic Engine	18
4.1	Tổng quan	18
4.2	Biểu diễn trạng thái hệ thống	18
4.3	Logic thời gian (Temporal Logic)	18
4.3.1	Linear Temporal Logic (LTL)	18
4.3.2	Ví dụ vi phạm vượt đèn đỏ	18
4.4	Mô hình hóa vi phạm tốc độ	19
4.4.1	Ràng buộc	19
4.4.2	Phát hiện vi phạm	19
4.4.3	Ổn định theo thời gian	19
4.5	Finite State Machine (FSM)	19
4.5.1	Định nghĩa	19
4.5.2	Ví dụ: vượt đèn đỏ	19
4.6	Constraint Satisfaction Problem (CSP)	20
4.7	Multi-violation detection	20

4.8	Đánh giá độ tin cậy	20
4.8.1	Voting theo thời gian	20
4.8.2	False positive control	20
4.9	Tối ưu hóa	20
4.9.1	Latency	20
4.9.2	Trade-off	21
4.10	Kết luận	21
5	Lớp 6 — Event Packaging và OTA	22
5.1	Tổng quan	22
5.2	Biểu diễn sự kiện	22
5.3	Tối ưu băng thông	22
5.3.1	So sánh	22
5.3.2	Compression	22
5.4	Edge vs Cloud	22
5.4.1	Phân chia	22
5.4.2	Latency	23
5.5	Giao thức truyền thông	23
5.5.1	Mô hình	23
5.5.2	Reliability	23
5.6	OTA (Over-the-Air)	23
5.6.1	Mô hình cập nhật	23
5.6.2	Rollback	23
5.7	Federated Learning (mở rộng)	23
5.8	Security	24
5.8.1	Encryption	24
5.8.2	Authentication	24
5.9	Tối ưu hệ thống	24
5.9.1	Bandwidth constraint	24
5.9.2	Event filtering	24
5.10	Kết luận	24

Lời nói đầu

Trong bối cảnh các hệ thống thị giác máy tính ngày càng được triển khai rộng rãi trong môi trường thực tế, đặc biệt là các bài toán yêu cầu xử lý thời gian thực với độ chính xác cao, việc thiết kế một pipeline xử lý từ cảm biến đến suy luận trở thành một thách thức mang tính liên ngành. Không chỉ dừng lại ở việc xây dựng các mô hình học sâu mạnh mẽ, hiệu năng tổng thể của hệ thống còn phụ thuộc chặt chẽ vào sự phối hợp giữa phần cứng, xử lý tín hiệu, tối ưu hóa tính toán và cơ chế ra quyết định.

Tài liệu này tập trung phân tích một kiến trúc hệ thống thị giác hiện đại theo cách tiếp cận phân lớp, trong đó mỗi lớp đảm nhiệm một vai trò riêng biệt nhưng có sự liên kết chặt chẽ về mặt dữ liệu và thời gian. Bắt đầu từ lớp cảm biến với các ràng buộc vật lý của ánh sáng và chuyển động, hệ thống tiến dần qua các lớp xử lý tín hiệu (ISP), tối ưu truyền dữ liệu (codec và trigger), suy luận AI, theo dõi đối tượng, và cuối cùng là lớp suy luận logic và tích hợp hệ thống ở mức ứng dụng.

Điểm trọng tâm của tài liệu không chỉ nằm ở việc mô tả kiến trúc, mà còn đi sâu vào các nguyên lý toán học và kỹ thuật nền tảng chi phối hoạt động của từng lớp. Từ các mô hình xác suất, tối ưu hóa, cho đến logic thời gian và hệ phân tán, mỗi thành phần đều được trình bày với mục tiêu làm rõ bản chất và các trade-off trong thiết kế hệ thống thực tế.

Thông qua cách tiếp cận này, tài liệu hướng tới việc cung cấp một cái nhìn toàn diện, có hệ thống và mang tính học thuật, giúp người đọc không chỉ hiểu *hệ thống hoạt động như thế nào*, mà còn nắm được *tại sao nó phải được thiết kế theo cách đó*.

Chương 1

Lớp 0 — Phần cứng và Cảm biến

1.1 Tổng quan vai trò của lớp cảm biến trong hệ thống thị giác tốc độ cao

Lớp phần cứng và cảm biến đóng vai trò nền tảng trong toàn bộ pipeline thị giác máy tính, đặc biệt trong các hệ thống nhận dạng phương tiện tốc độ cao. Đây là lớp quyết định trực tiếp đến chất lượng dữ liệu đầu vào, từ đó ảnh hưởng đến toàn bộ các lớp xử lý phía sau như ISP, AI inference, và tracking.

Không giống các bài toán thị giác thông thường, môi trường giao thông tốc độ cao đặt ra các yêu cầu cực kỳ khắt khe về:

- Độ sắc nét hình ảnh (sharpness)
- Độ chính xác hình học (geometric fidelity)
- Khả năng hoạt động trong điều kiện ánh sáng yếu
- Khả năng đóng băng chuyển động (motion freezing)

Các yếu tố này dẫn đến việc lựa chọn kiến trúc cảm biến và hệ thống chiếu sáng trở thành điều kiện tiên quyết.

1.2 Global Shutter và Rolling Shutter

1.2.1 Nguyên lý Rolling Shutter

Trong các cảm biến CMOS truyền thống, quá trình ghi nhận hình ảnh diễn ra theo cơ chế quét tuần tự từng dòng (row-wise readout). Điều này có nghĩa là mỗi hàng pixel được phơi sáng tại các thời điểm khác nhau.

Giả sử thời gian quét toàn bộ frame là $T_{readout}$, thì hàng thứ i sẽ được ghi nhận tại thời điểm:

$$t_i = t_0 + i \cdot \Delta t$$

Hệ quả:

- Các vật thể chuyển động nhanh bị biến dạng hình học
- Xuất hiện hiện tượng skew (ngiên), wobble (rung), hoặc shear (xé hình)

1.2.2 Nguyên lý Global Shutter

Ngược lại, cảm biến global shutter cho phép toàn bộ pixel trên sensor được phơi sáng đồng thời:

$$t_1 = t_2 = \dots = t_n$$

Điều này đảm bảo:

- Không có sai lệch thời gian giữa các pixel
- Hình ảnh giữ nguyên cấu trúc hình học của vật thể

1.2.3 So sánh định lượng

Xét một phương tiện di chuyển với vận tốc v :

$$v = 250 \text{ km/h} \approx 69.4 \text{ m/s}$$

Nếu thời gian quét của rolling shutter là 10 ms, thì sai lệch vị trí giữa dòng đầu và dòng cuối là:

$$\Delta x = v \cdot T_{readout} = 69.4 \cdot 0.01 = 0.694 \text{ m}$$

Sai lệch gần 70 cm là không thể chấp nhận trong bài toán nhận dạng biển số.

1.3 Motion Blur và yêu cầu thời gian phơi sáng

Ngay cả khi sử dụng global shutter, hiện tượng mờ chuyển động vẫn tồn tại do thời gian phơi sáng hữu hạn.

Độ dài vết mờ được xác định bởi:

$$d = v \cdot t_{exp}$$

Trong đó:

- d : độ dài vết mờ
- v : vận tốc vật thể
- t_{exp} : thời gian phơi sáng

1.3.1 Ràng buộc thiết kế

Để đảm bảo ký tự trên biển số có thể đọc được, yêu cầu:

$$d < 1 \text{ cm}$$

Suy ra:

$$t_{exp} < \frac{0.01}{69.4} \approx 144 \mu s$$

Điều này đặt ra yêu cầu:

- Shutter cực nhanh (trên 1/7000s)
- Sensor có độ nhạy cao

1.4 Giới hạn photon và bài toán ánh sáng

Khi giảm thời gian phơi sáng, số lượng photon thu được giảm theo:

$$N \propto E \cdot t_{exp}$$

Trong đó:

- N : số photon
- E : cường độ ánh sáng

Hệ quả:

- Ảnh bị tối
- Tăng nhiễu (shot noise, read noise)

Do đó, cần bổ sung nguồn sáng chủ động.

1.5 Hệ thống chiếu sáng hồng ngoại (IR Illuminator)

1.5.1 Nguyên lý hoạt động

Hệ thống sử dụng nguồn sáng hồng ngoại gần (Near-Infrared) để chiếu sáng cảnh:

- Bước sóng phổ biến: 850 nm và 940 nm
- Hoạt động độc lập với ánh sáng môi trường

1.5.2 So sánh 850nm và 940nm

Thuộc tính	850 nm	940 nm
Độ nhạy sensor	Cao	Thấp
Hiệu suất chiếu sáng	Mạnh	Yếu
Khả năng nhìn thấy	Có ánh đỏ	Gần như vô hình
Ứng dụng	Giao thông	An ninh kín đáo

1.5.3 Tương tác với vật liệu phản quang

Biển số phương tiện thường sử dụng vật liệu retroreflective, có đặc tính:

- Phản xạ ánh sáng ngược lại nguồn phát
- Tăng cường độ sáng tương đối so với nền

Khi chiếu IR:

- Biển số sáng rõ
- Nền xe tối hơn

1.6 Tổng hợp các ràng buộc thiết kế

Để đảm bảo hệ thống hoạt động chính xác ở tốc độ cao, cần đồng thời thỏa mãn:

- Global shutter: loại bỏ méo hình
- Exposure ngắn: giảm motion blur
- IR mạnh: đảm bảo đủ ánh sáng

Ba yếu tố này tạo thành một hệ ràng buộc:

$$\text{Image Quality} = f(\text{Shutter}, \text{Exposure}, \text{Illumination})$$

Thiếu bất kỳ thành phần nào:

- Không thể phục hồi bằng xử lý phần mềm
- Dẫn đến mất thông tin không thể đảo ngược

1.7 Kết luận chương

Lớp phần cứng không chỉ là tầng thu thập dữ liệu mà là yếu tố quyết định tính khả thi của toàn bộ hệ thống. Các sai lệch phát sinh tại đây mang bản chất vật lý, do đó không thể được bù đắp hoàn toàn bằng các mô hình học sâu ở các lớp phía sau.

Việc thiết kế đúng lớp cảm biến là điều kiện tiên quyết để đảm bảo:

- Độ chính xác nhận dạng
- Độ tin cậy hệ thống
- Khả năng vận hành trong môi trường thực tế

Chương 2

Lớp 2 — Video Codec và Cơ chế Trigger

2.1 Giới thiệu

Trong các hệ thống thị giác máy tính thời gian thực, việc xử lý toàn bộ các frame video bằng mô hình AI là không khả thi về mặt tính toán. Đặc biệt với các hệ thống đa luồng (multi-stream), số lượng frame cần xử lý mỗi giây có thể vượt xa năng lực của phần cứng suy luận (NPU).

Do đó, lớp Video Codec và Trigger được thiết kế nhằm:

- Giảm tải tính toán cho NPU
- Tối ưu hóa pipeline xử lý song song
- Lựa chọn các frame có giá trị thông tin cao nhất

Đây là lớp trung gian giữa xử lý tín hiệu hình ảnh (ISP) và suy luận AI (Inference Engine).

2.2 Tách biệt xử lý CPU và AI Inference

2.2.1 Mô hình pipeline truyền thống

Trong pipeline đơn giản:

Frame $\rightarrow$ Decode $\rightarrow$ AI Inference

Mọi frame đều được đưa qua AI, dẫn đến:

- Tải tính toán tuyến tính theo số frame
- Lãng phí tài nguyên với các frame dư thừa

2.2.2 Mô hình pipeline tối ưu

Hệ thống hiện đại sử dụng pipeline tách biệt:

Frame $\rightarrow$ Codec $\rightarrow$ CPU Preprocessing $\rightarrow$ Trigger $\rightarrow$ AI

Cho phép:

- CPU xử lý song song nhiều stream
- NPU chỉ xử lý subset frame

2.2.3 Mô hình hàng đợi (Queueing Model)

Xét hệ thống với:

- N luồng video
- Tốc độ mỗi luồng: f (fps)

Tổng số frame mỗi giây:

$$\lambda = N \cdot f$$

Nếu xác suất một frame được chọn bởi trigger là p , thì tải vào NPU là:

$$\lambda_{AI} = p \cdot \lambda$$

Mục tiêu:

$$\lambda_{AI} \ll \lambda$$

2.3 Video Codec và cấu trúc GOP

2.3.1 Group of Pictures (GOP)

Video được mã hóa dưới dạng các frame:

- I-frame (Intra): đầy đủ thông tin
- P-frame (Predicted): dựa vào frame trước
- B-frame (Bi-directional): nội suy từ hai phía

Chuỗi GOP có dạng:

$$I \rightarrow P \rightarrow B \rightarrow P \rightarrow \dots$$

2.3.2 Tối ưu hóa decode

Chỉ decode các frame cần thiết:

- Ưu tiên I-frame để lấy snapshot rõ nét
- Skip B-frame nếu không cần thiết

Giảm chi phí:

$$C_{decode} = \sum_{i \in \text{selected frames}} c_i$$

2.4 Cơ chế ROI Trigger

2.4.1 Định nghĩa ROI

Region of Interest (ROI) là vùng trong frame có khả năng chứa đối tượng cần quan tâm.

Giả sử frame $F(x, y)$, ROI là:

$$R = \{(x, y) \mid (x, y) \in \Omega\}$$

2.4.2 Trigger dựa trên biến đổi cường độ

Một trigger đơn giản dựa trên sự thay đổi:

$$\Delta I = \sum_{(x,y) \in R} |F_t(x, y) - F_{t-1}(x, y)|$$

Nếu:

$$\Delta I > \tau$$

→ kích hoạt inference.

2.4.3 Trigger dựa trên optical flow

Dùng trường chuyển động $\vec{v}(x, y)$:

$$E_{flow} = \sum_{(x,y) \in R} \|\vec{v}(x, y)\|$$

Trigger khi:

$$E_{flow} > \tau_v$$

2.5 Frame Selector và bài toán tối ưu

2.5.1 Định nghĩa hàm chất lượng frame

Mỗi frame được gán một score:

$$Q(F) = w_1 S(F) + w_2 C(F) + w_3 B(F)$$

Trong đó:

- $S(F)$: độ sắc nét (sharpness)
- $C(F)$: độ tương phản (contrast)
- $B(F)$: độ mờ chuyển động (blur penalty)

2.5.2 Đo độ sắc nét

Dựa trên gradient:

$$S(F) = \sum_{x,y} |\nabla F(x, y)|^2$$

2.5.3 Đo motion blur

$$B(F) = \int \left| \frac{\partial F}{\partial t} \right| dx dy$$

2.5.4 Bài toán chọn frame

Chọn frame tối ưu trong cửa sổ thời gian:

$$F^* = \arg \max_{F_i \in \mathcal{W}} Q(F_i)$$

2.6 Giảm tải NPU và tối ưu hệ thống

2.6.1 Hiệu suất tính toán

Giả sử:

- NPU xử lý tối đa K frame/s

Ràng buộc:

$$\lambda_{AI} \leq K$$

2.6.2 Tối ưu hóa xác suất trigger

Tìm p sao cho:

$$\max \text{Accuracy}(p) \quad \text{subject to} \quad p \cdot \lambda \leq K$$

Đây là bài toán trade-off:

- p thấp $\rightarrow$ mất sự kiện
- p cao $\rightarrow$ quá tải hệ thống

2.7 Song song hóa đa luồng

Hệ thống sử dụng kiến trúc pipeline song song:

$$\begin{aligned} \text{Stream}_1 &\rightarrow \text{CPU}_1 \rightarrow \text{Queue} \\ \text{Stream}_2 &\rightarrow \text{CPU}_2 \rightarrow \text{Queue} \\ &\vdots \\ \text{Queue} &\rightarrow \text{NPU Scheduler} \end{aligned}$$

Sử dụng chiến lược:

- Round-robin scheduling
- Priority-based scheduling

2.8 Kết luận chương

Lớp Video Codec và Trigger đóng vai trò tối ưu hóa tài nguyên trong hệ thống thị giác thời gian thực. Thay vì xử lý toàn bộ dữ liệu đầu vào, hệ thống chuyển sang xử lý có chọn lọc dựa trên thông tin.

Các đóng góp chính của lớp này:

- Giảm tải đáng kể cho NPU
- Tăng khả năng mở rộng (scalability)
- Duy trì độ chính xác thông qua lựa chọn frame tối ưu

Đây là bước chuyển quan trọng từ xử lý brute-force sang xử lý thông minh dựa trên thông tin.

Chương 3

Lớp 3 — Core AI Inference

3.1 Tổng quan

Lớp Core AI Inference là trung tâm tính toán của toàn bộ hệ thống, nơi thực hiện các tác vụ nhận thức thị giác như phát hiện đối tượng, nhận dạng ký tự và ước lượng trạng thái động học. Lớp này thường được triển khai trên các phần cứng chuyên dụng như NPU (Neural Processing Unit), DSP hoặc GPU tích hợp.

Pipeline suy luận bao gồm ba thành phần chính:

- Phát hiện phương tiện (Vehicle Detection)
- Nhận dạng biển số (Sequence Recognition)
- Ước lượng tốc độ (Speed Estimation)

3.2 Bài toán phát hiện đối tượng

3.2.1 Mô hình hóa

Cho ảnh đầu vào $I \in \mathbb{R}^{H \times W \times 3}$, mục tiêu là tìm tập các bounding box:

$$\mathcal{B} = \{(x_i, y_i, w_i, h_i, c_i)\}_{i=1}^N$$

Trong đó:

- (x_i, y_i) : tọa độ tâm
- (w_i, h_i) : kích thước
- c_i : confidence score

3.2.2 Anchor-free detection

Thay vì sử dụng anchor boxes, mô hình dự đoán trực tiếp:

$$P(x, y) = (p_{obj}, t_x, t_y, t_w, t_h)$$

Loss function:

$$\mathcal{L} = \lambda_1 \mathcal{L}_{cls} + \lambda_2 \mathcal{L}_{bbox} + \lambda_3 \mathcal{L}_{IoU}$$

3.2.3 IoU Loss

$$IoU = \frac{B_{pred} \cap B_{gt}}{B_{pred} \cup B_{gt}}$$

$$\mathcal{L}_{IoU} = 1 - IoU$$

3.3 Pipeline nhận dạng chuỗi ký tự

3.3.1 Bài toán

Cho feature map $F \in \mathbb{R}^{T \times D}$, cần ánh xạ thành chuỗi ký tự:

$$Y = (y_1, y_2, \dots, y_n)$$

3.3.2 CTC (Connectionist Temporal Classification)

Xác suất chuỗi:

$$P(Y|F) = \sum_{\pi \in \mathcal{B}^{-1}(Y)} \prod_{t=1}^T P(\pi_t | F_t)$$

Trong đó:

- π : alignment path
- $\mathcal{B}$: hàm loại bỏ blank và lặp

Loss:

$$\mathcal{L}_{CTC} = -\log P(Y|F)$$

3.3.3 Ưu điểm

- Không cần segmentation ký tự
- Giảm lỗi lan truyền (error propagation)
- Tối ưu latency

3.4 Ước lượng tốc độ

3.4.1 Chuyển đổi tọa độ

Giả sử có hai điểm GPS:

$$P_1 = (\phi_1, \lambda_1), \quad P_2 = (\phi_2, \lambda_2)$$

Khoảng cách theo công thức Haversine:

$$d = 2R \cdot \arcsin \left(\sqrt{\sin^2 \left(\frac{\Delta \phi}{2} \right) + \cos(\phi_1) \cos(\phi_2) \sin^2 \left(\frac{\Delta \lambda}{2} \right)} \right)$$

3.4.2 Tốc độ

$$v = \frac{d}{\Delta t}$$

3.5 Tối ưu hóa trên NPU

3.5.1 Quantization

Giảm độ chính xác:

$$FP32 \rightarrow INT8$$

Sai số:

$$\epsilon = |x - \hat{x}|$$

3.5.2 Throughput

Nếu mỗi inference mất t_{inf} :

$$Throughput = \frac{1}{t_{inf}}$$

3.6 Kết luận

Lớp inference là nơi chuyển đổi dữ liệu hình ảnh thành thông tin ngữ nghĩa. Hiệu năng của lớp này phụ thuộc mạnh vào tối ưu phần cứng và thiết kế mô hình.

iết tiếp chương 5 theo kiểu hardcore research

Chương 4

Lớp 5 — Violation Logic Engine

4.1 Tổng quan

Lớp Violation Logic Engine thực hiện suy luận cấp cao trên dữ liệu đã được xử lý từ các lớp trước. Đây là lớp chuyển đổi từ thông tin cảm nhận (perception) sang quyết định (decision), dựa trên các quy tắc logic, ràng buộc không-thời gian (spatio-temporal constraints) và ngữ cảnh.

Khác với các lớp AI dựa trên học máy, lớp này chủ yếu sử dụng:

- Rule-based reasoning
- Temporal logic
- Constraint satisfaction

4.2 Biểu diễn trạng thái hệ thống

Tại thời điểm t , trạng thái của một đối tượng được biểu diễn:

$$s_t = (x_t, y_t, v_t, id, lane_t, signal_t)$$

Toàn bộ hệ thống:

$$S = \{s_t^1, s_t^2, \dots, s_t^N\}$$

4.3 Logic thời gian (Temporal Logic)

4.3.1 Linear Temporal Logic (LTL)

Sử dụng các toán tử:

- G : luôn luôn (Globally)
- F : tồn tại trong tương lai (Eventually)
- X : trạng thái kế tiếp
- U : cho đến khi

4.3.2 Ví dụ vi phạm vượt đèn đỏ

$$G(\text{signal} = RED \rightarrow \neg \text{cross_line})$$

Vi phạm khi:

$$F(\text{signal} = RED \wedge \text{cross_line})$$

4.4 Mô hình hóa vi phạm tốc độ

4.4.1 Ràng buộc

$$v_t \leq v_{max}$$

4.4.2 Phát hiện vi phạm

$$\exists t : v_t > v_{max}$$

4.4.3 Ổn định theo thời gian

Để tránh nhiễu:

$$\frac{1}{T} \int_t^{t+T} v(\tau) d\tau > v_{max}$$

4.5 Finite State Machine (FSM)

4.5.1 Định nghĩa

FSM:

$$M = (Q, \Sigma, \delta, q_0, F)$$

- Q : tập trạng thái
- Σ : input events
- δ : hàm chuyển trạng thái

4.5.2 Ví dụ: vượt đèn đỏ

- q_0 : trước vạch
- q_1 : gần vạch
- q_2 : vượt vạch

Transition:

$$\delta(q_1, \text{signal} = RED \wedge \text{cross}) = q_{violation}$$

4.6 Constraint Satisfaction Problem (CSP)

Biểu diễn vi phạm như bài toán:

Find x such that $C(x) = true$

Ví dụ:

$$C = (lane = forbidden) \wedge (direction = wrong)$$

4.7 Multi-violation detection

Xét tập luật:

$$\mathcal{R} = \{r_1, r_2, \dots, r_k\}$$

Hệ thống đánh giá:

$$V = \{r_i(s_t)\}$$

4.8 Đánh giá độ tin cậy

4.8.1 Voting theo thời gian

$$score = \sum_{t=1}^T \mathbb{I}(\text{violation}_t)$$

Quyết định:

$$score > \tau$$

4.8.2 False positive control

$$FP = \frac{\text{False detections}}{\text{Total detections}}$$

Mục tiêu:

$$FP \rightarrow 0$$

4.9 Tối ưu hóa

4.9.1 Latency

$$T_{total} = T_{perception} + T_{logic}$$

4.9.2 Trade-off

- Logic phức tạp $\rightarrow$ chính xác cao
- Nhưng tăng latency

4.10 Kết luận

Violation Engine là lớp suy luận quyết định, sử dụng các công cụ hình thức để đảm bảo tính đúng đắn và độ tin cậy cao trong môi trường thực tế.

Chương 5

Lớp 6 — Event Packaging và OTA

5.1 Tổng quan

Lớp cuối cùng chịu trách nhiệm:

- Nén và đóng gói dữ liệu
- Truyền thông edge-cloud
- Cập nhật mô hình từ xa (OTA)

Mục tiêu:

- Giảm băng thông
- Tăng khả năng mở rộng
- Đảm bảo cập nhật liên tục

5.2 Biểu diễn sự kiện

Một event được định nghĩa:

$$E = (I_{crop}, plate, t, gps, v, type)$$

5.3 Tối ưu băng thông

5.3.1 So sánh

- Raw video: $\sim$ Mbps
- Event: $\sim$ KB

5.3.2 Compression

$$R = \frac{\text{Original size}}{\text{Compressed size}}$$

5.4 Edge vs Cloud

5.4.1 Phân chia

Edge : *inference + filtering*

Cloud : *storage + analytics*

5.4.2 Latency

$$T = T_{edge} + T_{network} + T_{cloud}$$

5.5 Giao thức truyền thông

5.5.1 Mô hình

Device $\rightarrow$ Broker $\rightarrow$ Backend

5.5.2 Reliability

$$P_{success} = 1 - P_{loss}$$

5.6 OTA (Over-the-Air)

5.6.1 Mô hình cập nhật

$$\theta_{new} = \theta_{old} + \Delta\theta$$

5.6.2 Rollback

$$\theta = \begin{cases} \theta_{new} & \text{if success} \\ \theta_{old} & \text{if failure} \end{cases}$$

5.7 Federated Learning (mở rộng)

$$\theta = \sum_{i=1}^N w_i \theta_i$$

5.8 Security

5.8.1 Encryption

$$C = Enc(K, data)$$

5.8.2 Authentication

$$Auth = HMAC(K, message)$$

5.9 Tối ưu hệ thống

5.9.1 Bandwidth constraint

$$B_{used} \leq B_{max}$$

5.9.2 Event filtering

$$E_{sent} = \{E_i | score_i > \tau\}$$

5.10 Kết luận

Lớp Event và OTA giúp hệ thống hoạt động hiệu quả ở quy mô lớn, đảm bảo khả năng cập nhật và tối ưu liên tục mà không cần can thiệp vật lý.